

CoSc 3311

Fundamentals of Software Engineering

Dagmawi Lemma (PhD)

Assistant Professor, AAU

courses.dag@gmail.com

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

1

2. Requirements Elicitation

CoSc3311 – Fundamentals of Software Engineering

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

53

Requirements Elicitation

- What is Requirement?
 - IEEE definition (610.12)
 - “1. a condition or capability needed by a user to solve a problem or achieve an objective.”
 - What the user needs to satisfy the responsibility of the role in the system
 - “2. a condition or capability that must be met or possessed by a system or system component to satisfy a contract, standard, specification, or other formally imposed document.”
 - What the environment needs
 - What the task demands

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

54

Requirements Elicitation

- NOTE: it is not about what the customer or user wants!
 - It is not personal need rather it is system-wide need
- Requirements are
 - Specification
 - of what should be implemented
 - defining how the system should behave
 - Constraints on the development of the software

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

55

Requirement Type

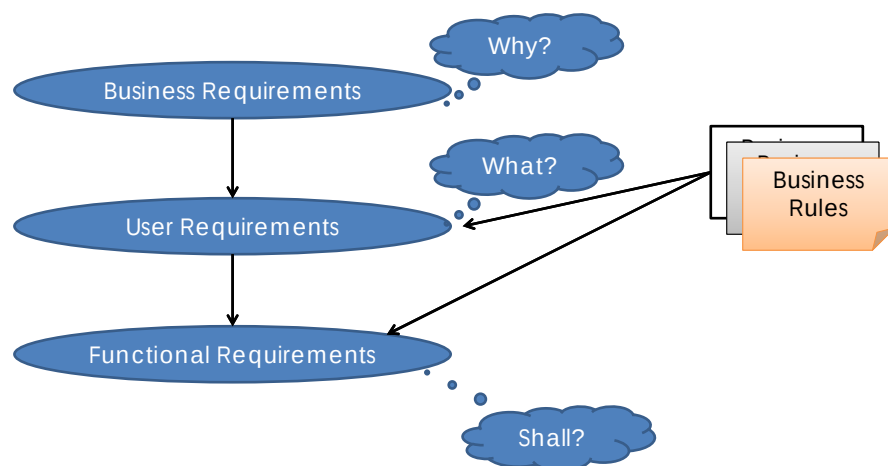
- Business Requirements
 - Why?
 - *Documented as vision and scope document*
- User Requirements
 - What?
 - What would the user be able to do with this system
 - use-cases are useful to describe requirements
 - General description of functional requirements
 - *Documented as use-case*
- Functional Requirements
 - What to be implemented
 - Can be derived using the use-case
 - *Documented as Software Requirement Specification (SRS)*

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

56

Requirement Type



Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

57

Requirement Type

- Primarily - the business requirement defines why the software is needed, but also defines
 - The features needed
 - The business rules to be respected
 - E.g. Library system:
 - The business requirement defines why the software (online system) is required, what would be the scope of the system, etc.
 - members can reserve books online (The business requirement → improved satisfaction)

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

58

Requirement Type

- Business Rules
 - are principles, policies, guidelines, standards, etc. to be respected when performing a task or to be part of the system
 - Defines
 - Rules to be respected as functional requirements are executed
 - Rules to be respected to meet the business requirements and the quality attributes
 - Examples
 - in a university-library,
 - a *member* should be active student or staff member of the university
 - » *This requirement shows the business rules to be satisfied in order to be member of the library (active "student" or "staff members")*
 - a *member* should return the book he/she borrowed within 2weeks (14days)
 - *This requirement shows the business rules to be respected after a book is borrowed*
 - Can be associated with various types of requirements also used to identify and clarify requirements

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

59

Requirement Type

- Business Rules (continued)
 - functional
 - Examples
 - the system shall record the **book-barcode**, *return-date* and the *member-ID* in the borrowed list when a member borrow book from the library
 - the system shall record the **ISBN**, *reservation-date*, *reservation-time* and the *member-ID* in the reserved list when a member reserve a book in the library
 - Nonfunctional
 - Examples
 - the system shall allow a member to borrow **a maximum of four books**
 - the system shall allow a member to reserve **a maximum of one book**
 - the record made during the reservation of book shall be turned into borrowed list **in 48hours** otherwise the record shall be deleted
 - pseudo-requirements
 - Examples
 - the online system shall be built using php and MySQL database

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

60

Requirement Type

- Requirement can be generally classified as functional or nonfunctional
 - Functional focus on what task to be done
 - Nonfunctional explains the **customizable** business rules (e.g. password length, response time, etc.) to be maintained while doing the task and the required environment
 - Business rules
 - System requirements which further describes features of the product

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

61

Requirement Type

- NFR
 - Business rules
 - Corporate policies
 - E.g.
 - » how frequently the user shall change the password
 - » Password length
 - Industry standards
 - Government policies
 - Quality attributes (the “ilities”)
 - Usability
 - Maintainability
 - Dependability
 - Portability
 - As well as performance, cost etc.

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

62

Good Characteristics of Requirement

- How do we know we have good requirement?
 - Verification
 - Are we building the product right?
 - refers to the set of activities that ensure that software (the product) correctly implements a specific function
 - What we implement, is it correct?
 - Validation
 - Are we building the right product?
 - refers to a different set of activities that ensure that the software that has been built is traceable to customer requirements
 - Did we implement what the user requires?

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

63

Required Characteristics

- **Correctness**
 - A specification is correct if it represents the client's view of the system.
 - Each requirement shall describe something actually needed by the customer; and that is related to the business requirement.
 - The model describes the reality of interest to the client, not another reality.

Example Business Requirement:

- To improve customer satisfaction by providing features like remote book reservation and searching.

Example User Story:

- As a Librarian I want to record books borrowed by members in the borrow list so that I can trace who borrowed which book

The example user story is true and it may support the user's task (Librarian's), but is it valid in view of the business requirement?

Perhaps it needs to be corrected as:

- As a Librarian I want to record books borrowed by members in the borrow list so that I can easily have information on the book inventory and the availability of a book

Required Characteristics

- **Consistency**
 - Should not contradict itself
 - All concepts in the model correspond to phenomena of the same reality
- **Completeness**
 - Every phenomenon of interest is described in the model by a concept
 - Externally - all desired properties are present.
 - Internally - no undefined references
- **Clarity**
 - Properties and items in the system shall be clearly described
 - All concepts in the model correspond to exactly one phenomenon
- Requirements shall be verifiable (testable)
- Requirements shall be traceable
- **Feasible and Realistic**
 - The model describes a reality that can exist
 - The requirement shall be implementable in the context of the environment

Required Characteristics

- Example
 - What is wrong with this requirement?
 - What is the Cataloguer?
 - Person, Hardware or Software ...
 - “required information of the book” ?
 - Type and detail of information shall be described

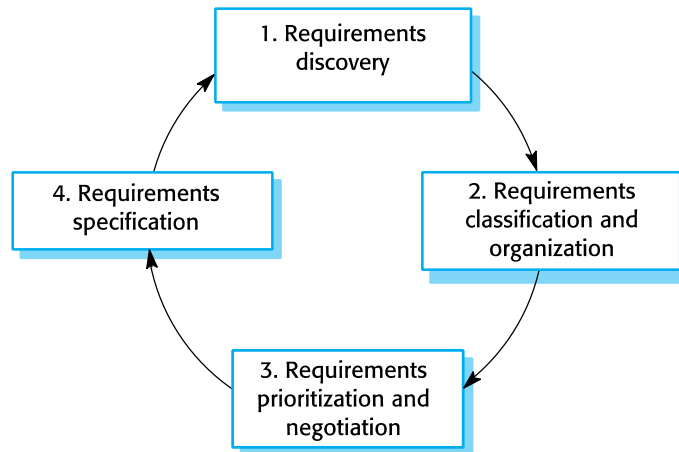
Identified Requirement
in the Catalogue Section
Role Identified: Cataloguer

Role Description: The Cataloguer
records required information of
the book.

Required Characteristics

- Example (continued)
 - Identified Requirement in the Catalogue Section
 - Role Identified: Cataloguer
 - Process Identified: Cataloging
 - Role Description: The Cataloguer is a person who is responsible to record new book information in the database and produce catalogue card. The required book information comprise ISBN, Book Title, Name of Author, Publisher, Barcode and Price of the book.
 - The ISBN is unique code assigned to a book, which consists Latin character and digits.
 - The Barcode is produced after the catalogue card is produced
 - The Catalogue card is produced in three forms ...

Elicitation and Analysis Process



Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

68

Elicitation and Analysis Process

- Requirements Elicitation:
 - Definition of the system in terms understood by the customer (*“Problem Description”*)
- Requirements Analysis:
 - Technical specification of the system in terms understood by the developer (*“Problem Specification”*)

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

69

Tasks in Elicitation

- Identifying actors/roles
 - Identifying scenarios
 - Identifying use-cases
 - Refining use-cases
 - Identifying relationships among use-cases
 - Identifying participating objects
 - Identifying nonfunctional requirements
- *The tasks and the order may vary based on the method*
 - *Are we following object Oriented approach or ...?*
 - *User centered or task centered or data centered*
 - *Note that this determines what to identify first (i.e., the class or the actor or the use-case or ...)*
 - *Not that one factor to select among the various method is the type of system you are developing (i.e., DSS or TPS or MIS or ...)*
 - *Scenario based or not ...?*

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

70

Requirement Identification

- For example [Brugge] present a method that
 - focus on scenario-based requirements elicitation
 - elicit requirements by observing and interviewing users, so that need to identify the users first
 - then represent the user's current work processes as as-is scenarios and then develop visionary scenarios describing the functionality provided by the future system

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

71

Requirement Identification

- Requirement Discovery / Determination / Identification
 - Plan how to do it
 - Gather Information
 - Review Identified Requirements
 - Prototype
 - Development and evaluation
 - JAD

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

72

Requirement Identification

- Background Analysis
 - Learn about the setting, the existing system and the physical processes related to the revised system
 - Who runs what
 - Who reports to whom
 - What is relationship between x and y?
 - E.g. relationship between catalogue section and circulation desk – in a library system
 - The nature and the type of interaction between processes
 - Etc.

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

73

Requirement Identification

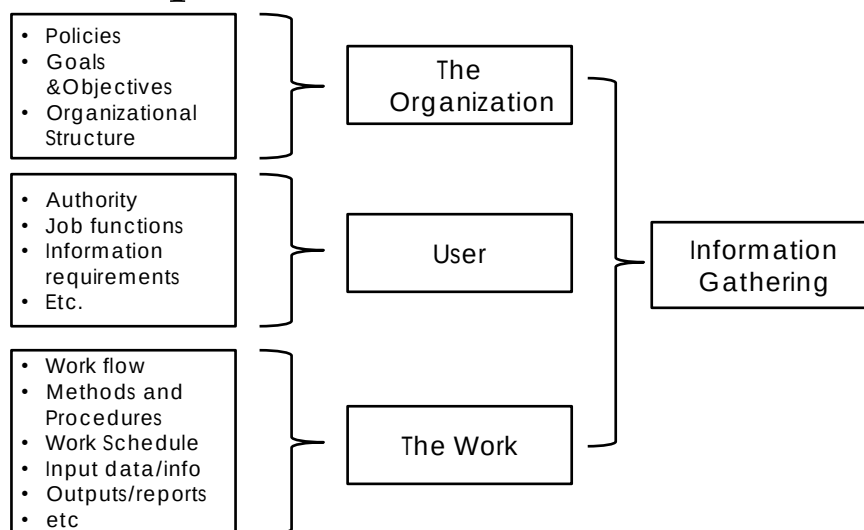
- Information Gathering
 - What kind of information do we need?
 - About the firm
 - About the users
 - About the process
 - Techniques
 - Site Observation and Interview
 - Document Review/Investigation
 - Interview
 - Questionnaire

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

74

Requirement Identification



Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

75

Requirement Identification

- Traditional Methods for Requirement Identification
 - Interview
 - Questionnaire
 - Site/User observation
 - Document Review

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

76

Requirement Identification

1. Interview

- Guideline
 - Plan the Interview
 - Identify interviewee: appointment, lead questions
 - Think about what you need to find out
 - Listen carefully and take notes (tape record if permitted)
 - Review notes within 48hrs of the interview
 - Be neutral
 - Seek diverse views
- Interview Questions
 - Open-ended questions
 - Close-ended questions
- **Thank the Interviewee** at the beginning and at the end

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

77

Requirement Identification

2. Questionnaires

– Choosing respondents

- if more people to survey decide which set of people to send questionnaire to or which questionnaire to send to which group of people
 - Convenient: people at a local site or willing to get surveyed
 - Random sample: select any person from a list
 - Purposeful sample: select people who satisfy certain criteria
 - Stratified sample: select random set from each of many categories

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

78

Requirement Identification

	Questionnaires	Interview
Useful for ...	Specific purpose	General information gathering
Type of questions	Close-ended	Open-ended as well as Close-ended
Analysis	Easy	Difficult
Clarity	Has to be extremely clear	Can be clarified on-site
Time	Required less time	Requires more time
Participant	Could involve many respondent	Could only involve only representatives of the target group

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

79

Requirement Identification

3. Site/User Observation

- People cannot always be trusted to reliably report their own actions
 - They may think some details are commonsense since they have done it for a very long time
 - They are afraid of the consequence of their answers
- Often difficult to obtain unbiased data
- People often work differently when being observed
 - Telling the task manual but doing something else
- Formal systems Vs Informal system

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

80

Requirement Identification

4. Document Review

- Four types of useful documents
 - Written work procedure for an individual or a work group
 - Describes how a job is performed
 - Includes data and information used and created in the process of performing the job or task
 - Business form
 - Explicitly indicate what data flow in or out of a system and which are necessary for the system to work
 - Report generated by current systems
 - Enables the analyst to work backwards from the report to the data that generated it – company's performance is past years
 - Description of current information system
 - If the current system is computer based

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

81

Requirement Identification

- Modern Methods for Requirement Identification
 - Joint Application Design (JAD)
 - Prototype
 - CASE tools

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

82

Requirement Identification

1. JAD

- Similar to group interview as it brings together key users, managers and systems analysts
- Purpose: collect system requirements simultaneously from key people
- Particular structure of roles and agenda is followed and analysts control the sequence of questions answered by users
- Conducted off-site to keep participants away from distractions
- may last from four hours to an entire week and may consist of many weeks

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

83

Requirement Identification

1. JAD (continued)

– Participants

- Session Leader – organizes and runs the JAD
- Users – key users of the current system
- Managers of the workgroups who use the current system
- Sponsor – needed to cover expenses
- Systems Analysts – to learn from users and managers
- Scribe – takes notes
- IT Staff – other IT staff like programmers, database analysts

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

84

Requirement Identification

2. Prototyping

- Repetitive process
- Rudimentary version of system is built
- Replaces or augments SDLC
- Goal: to develop concrete specifications for ultimate system

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

85

Requirement Identification

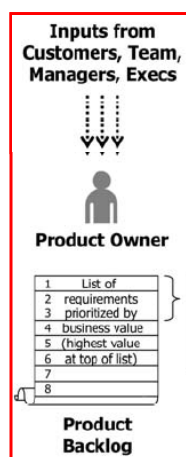
- In the case of waterfall models, requirement identification focus on “what” is needed by the “system”, or “what” the “system” shall fulfill.
 - hence **scenarios** can be used to identify, primarily, the system’s requirements
- In the case of agile methods (scrum), instead of the system’s requirements, the focus is on the users’ requirements.
 - Hence **user stories** are used to identify, primarily, “what” and “why” the user demands a certain functionality.
 - A user story is the smallest unit of work with an end goal,
 - a functionality expected by end user or the customer.

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

86

Requirement Identification



- The purpose of a user story is to articulate how a piece of work will deliver a particular value back to the customer (stakeholders).

- In scrum, the **Product Owner** is responsible to collect the user stories and prioritize them as a **Product Backlog (whish list)**

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

87

Requirement Identification

- If we are following scrum, user stories are the requirements which should be identified from roles (actors) in the system.
 - However, actor is not necessarily human user but it can also be:
 - External system
 - Device component
- User stories shall be related and contextualized to a particular scenario (use case)

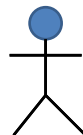
Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

88

Actors

- The **actor** in your system shall be considered as roles outside the boundary of the system who are initiating a certain action (use-case) or communicated by the use-case
- Actor is denoted by a sticky person



Actor ID Goes Here

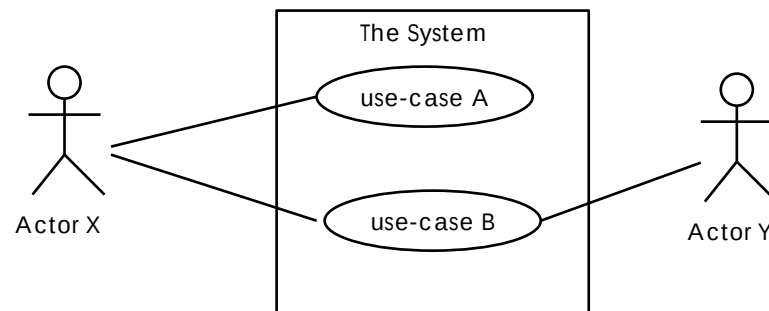
Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

89

Actors

- Note that use-cases are within the boundary of the system while actors are outside the boundary
- An actor can also be involved in more than one use-cases



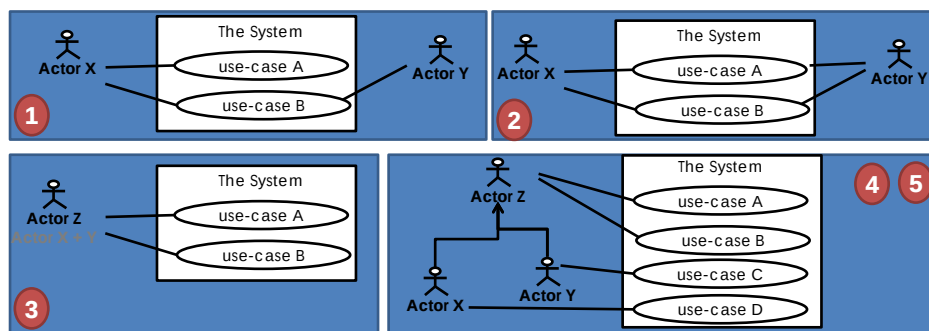
Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

90

Actors

- Identifying actors is an iterative process, just like many of the SE tasks
- For Example,
 1. Let's assume that you have identified more than one actor (Actor X and Y)
 2. Eventually you may noted that all the actors are participating in the same way
 3. Therefore you need to represent those actors using common name (Actor Z) = Actor X + Y
 4. In the process you may identify a new use-case in which only one of the merged actor is participating
 5. Therefore you need to have specialization of the actor
 6. And so on...



Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

91

Actors

- NOTE: most of the time actors are aligned to a human users, but actor can also be
 - an external system, which you strategically decided to be external as you define your use-case
 - a device, which perform a certain action but often a reason to initiate another use-case

Scenario

- “A narrative description of what people do and experience as they try to make use of computer systems and applications” [M. Carrol, Scenario-based Design, Wiley, 1995]
- Scenario can be
 - As-is
 - Visionary
 - Test / evaluation
 - training
- A concrete, focused, informal description of a single feature of the system used by a single actor.
- Scenarios can have many different uses during the software lifecycle
 - *Requirements Elicitation*: As-is scenario, visionary scenario
 - *Client Acceptance Test*: Evaluation scenario
 - *System Deployment*: Training scenario.

Example

- Scenario

- reserve_book (Main scenario)
 - expired_membership_reserve_book(Exception)
 - print_reservation_code (Extension)
- borrow_book (Main scenario)
 - borrow_for_spot_reading (Extension)
 - borrow_for_takeaway (Extension)

Possibly appears as
alternative flow
of use-case

Possibly appears as
generalization of use-
case

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

94

Heuristics: How to find use-cases

- Select a narrow vertical slice of the system (i.e. one scenario)
 - Discuss it in detail with the user to understand the user's preferred style of interaction
- Select a horizontal slice (i.e. many scenarios) to define the scope of the system.
 - Discuss the scope with the user
- Use illustrative prototypes (mock-ups) as visual support
- Find out what the user does
 - Task observation (Good)
 - Questionnaires (Bad)

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

95

Use-case

- If scenario based approach is followed, then the use-case is aggregation of scenario
- But a single scenario also can form the use-case



Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

96

Use-case

- Use-case is defined as the internal part of the system having a specific goal to achieve
 - A use-case does specific task
- Note that a certain actor can be involved into more than one use-cases
 - Whether the use-cases are sequentially dependent or not, it is recommended to associate a use-case with specific goal

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

97

Use-case

- We have seen that a use-case is represented in UML using oval shape having the name inside the shape
- Several use cases collectively build the system can be depicted with a use-case diagram
- Yet each use-case need to be described
 - No specific rule on how the use-case description should be organized
 - It can be narrated in a paragraph or two
 - It can be presented in structured form

Use-case

- The use-case description need to include and specify
 - Use-case ID,
 - which facilitate traceability to requirement/s
 - Use-case name
 - Goal
 - Participating actor/s
 - Actor that initiate the use-case
 - Actor that might be communicated by the use-case
 - Flow of event
 - Pre-condition (i.e., entry condition)
 - What should be fulfilled in order to start the use-case
 - Exception (alternative flow)
 - Post-condition (exit condition),
 - when do you say the use-case is done?
 - What should be fulfilled when the use-case is completed

Use-case Refinement and R/P

- use-case refinement
 - Validation and verification shall be done together with the user – actually by the user/owner of the system
 - Validate
 - Are we building the right system?
 - Have we identified the right use-case?
 - Have we included what the user needs?
 - Verify
 - Are we building the system right?
 - Have we used the correct terminologies?
 - Have we followed the correct procedure (flow of event)?
- Use-case Relationship
 - Include / use
 - Extend
 - Generalization/ Specialization

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

100

Use-case Refinement and R/P

- Use-case Relationship
 - It is worth to identify the possible relationship or dependencies between use-cases as part of the refinement process
 - For instance, we can avoid redundant descriptions which are appearing in the several use-cases we identified in the process
 - Avoiding redundancy helps to achieve consistency

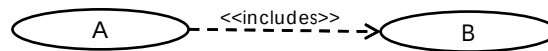
Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

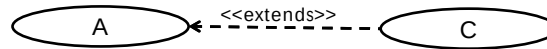
101

Use-case Refinement and R/P

- Include / use
 - Represented by using the include or use stereotype
`<<includes>>` or `<<uses>>`
 - This type of dependency or relationship is used to indicate that the main use-case (A) **needs** the flow of events described in the included use case (B) in order to perform and achieve its goal **or else** those flow of events should be listed in the main use case



- Extend
 - Represented by using the include or use stereotype
`<<extend>>`
 - This type of dependency or relationship is used to indicate that the main use-case (A) may benefit from the flow of events described in the extended use- case (C) while executing to achieve its goal



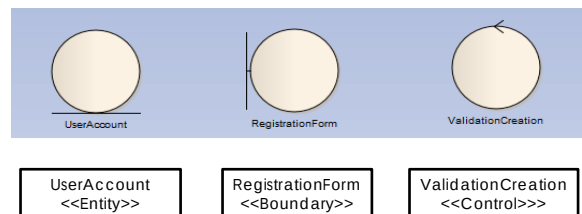
Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

102

Object/Class

- Identifying objects from use-case
 - The three type of objects



Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

103

Object/Class

- Boundary Class
 - Represents the interfaces between the actors and the system
 - used to model interaction between the system's surroundings and its inner workings
 - Common boundary classes include windows, communication protocols, printer interfaces, sensors, and terminals

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

104

Object/Class

- Entity Class
 - Manages the information the system need to provide the required functionality
 - used to model information and associated behavior that must be stored
 - **not specific to one** use-case realization

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

105

Object/Class

- Control Class
 - Represents the use-case logic and coordinates the other classes
 - Separates the interface classes from the business logic classes
 - Useful to decouple the dependency between boundary and entity class
 - provide coordinating behavior in the system